Team Name : (your team)

Team Leader Name :

Problem Statement : Rank the top 100 of 100,000 candidates for one nuanced Senior AI Engineer JD with recruiter-grade judgment — not keyword matching — avoiding keyword-stuffer traps and ~80 planted honeypots. Project: Lighthouse — a reasoning-first, explainable candidate ranker.

The Problem

- 100,000 candidates; genuine Senior-AI-Engineer fits are rare — only 0.78% even have an AI-ish title.

• Seeded traps: keyword-stuffers (an Accountant listing 9 AI skills), plain-language Tier-5s (real systems, no buzzwords), behavioral twins, and ~80 honeypots (subtly impossible profiles).

• The provided sample_submission ranks the stuffers #1–20 — the exact wrong answer. >10% honeypots in the top-100 = disqualification.

• Scoring is NDCG@10-heavy: the top-10 must be right. The real task is reasoning about the gap between what the JD says and what it means.

Solution Overview

- Lighthouse — an offline-LLM-augmented hybrid ranker: recruiter-grade, reasoning-first, fully explainable.
- Five fit components (semantic + role-coherence + career-evidence + experience + skill-trust), gated by JD hard-negatives and a behavioral modifier, with honeypots zeroed.
- Reads career evidence & semantics, not keywords — surfaces plain-language Tier-5s and sinks keyword-stuffers.
- Every pick ships a grounded, no-hallucination reason.
- CPU-only, < 5 min, no API at rank-time — production-shaped, not a GPT-per-candidate demo.

JD Understanding & Candidate Evaluation

- Must-haves: production embeddings/retrieval, vector/hybrid search ops, ranking-eval (NDCG/MRR/MAP), strong Python. Ideal 6–8 yrs, 4–5 applied-ML at product (not services) cos, shipped a ranking/search/recsys system.
- Hard-negatives encoded as gates: services-only, outside-India & won't-relocate (no visa), research-only, CV/speech-only w/o NLP, LangChain-only-recent, title-chasers, non-engineering current role.
- Signal priority: career-history evidence > job title > listed skills. Behavioral signals decide actual availability.
- Claude (offline) parsed the JD into a static committed rubric — read at rank-time with no API call.

Ranking Methodology

- Retrieve: precomputed bge-small embeddings + BM25 over clean per-candidate text blobs (100K).
- Score, each in [0,1]: semantic_fit (cosine vs 10 JD facets), role_coherence (taxonomy+semantic), career_evidence (built ranking/search at product cos), experience_fit (soft 6–8 curve), trust_skills (proficiency×duration×endorsements×assessment).
- Combine: weighted base (role_coherence 0.26 & career_evidence 0.24 lead) × Π(hard-negative gates) × behavioral modifier [0.80–1.12]; honeypots → 0.
- final = base × gates × behavior. Order by score desc; ties → candidate_id ascending; take top 100.

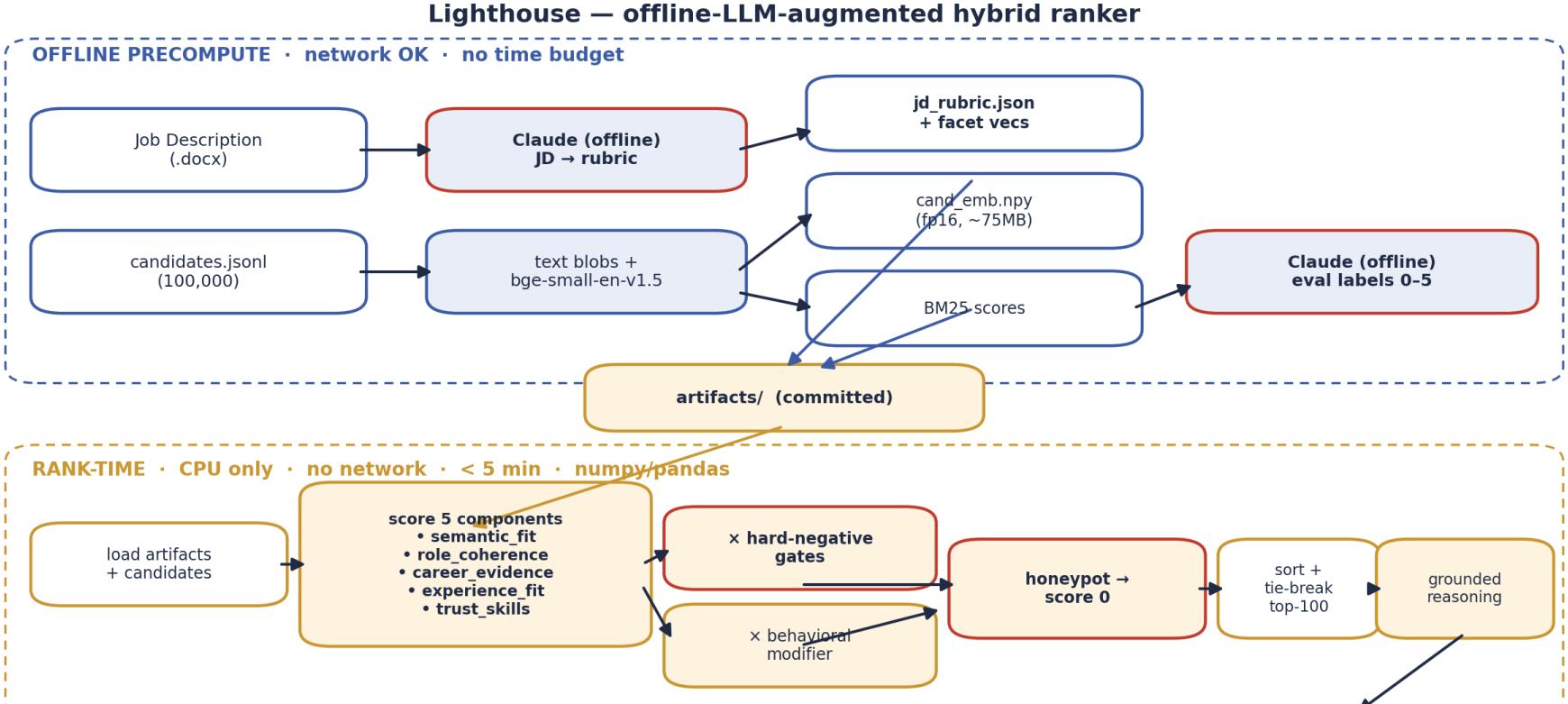
Explainability & Data Validation

- Deterministic, fact-grounded reasoning per candidate — cites real years, title, named skills, employers, signal values, and names honest gaps (e.g., '120-day notice; Toronto-based — relocation risk').
- Zero hallucination: every term traces to the candidate's own profile (enforced by tests). Variation by dominant factor; tone tracks the rank band.
- Honeypot/anomaly detector (explainable rules): ≥3 advanced/expert skills with 0 months, date contradictions, tenure overflowing experience, invalid education — flags 0.042% of the pool, no false positives on real profiles.
- Schema-tolerant parsing; sentinels (-1 github/offer, empty assessments — 60–76% of pool) treated as neutral, never penalized. Fully seeded/deterministic.

End-to-End Workflow

- Phase 1 — Offline precompute (network OK, no time limit): JD →(Claude)→ jd_rubric.json; candidates → text blobs → bge-small embeddings (fp16) + BM25; Claude → eval labels. Artifacts committed.
- Phase 2 — Rank-time (CPU, no network, < 5 min): load artifacts → score 5 components → gates × behavioral → honeypot zero → sort/tie-break → top-100 → grounded reasoning → submission.csv.
- Single reproduce command: python rank.py --candidates ./data/candidates.jsonl --out ./submission.csv → passes the official validator.

System Architecture



Results & Performance

- vs naive keyword baseline (Claude-authored proxy labels, 221-candidate stratified set): composite 0.998 vs 0.553; NDCG@10 1.000 vs 0.577; MAP 0.998 vs 0.438.
- Trap resistance: the keyword baseline puts 13/32 keyword-stuffers in its top-25; Lighthouse admits 0. Honeypots in top-100: 0 (DQ threshold >10%).
- Ablation: defenses are layered (role_coherence component + gate both fight stuffers); removing the whole anti-trap stack drops composite to 0.984 and lets honeypots climb (median rank 209→144).
- Constraints met: rank step CPU-only, no network, ~56 s (well under the 5-min budget) over 100K; embeddings precomputed (~75 MB fp16); 36 tests green.

Technologies Used

- sentence-transformers BAAI/bge-small-en-v1.5 — small, CPU-friendly, strong retrieval quality; precomputed so rank-time needs no model.
- rank_bm25 — lexical recall signal complementing dense retrieval.
- numpy / pandas — the entire rank step: fast, deterministic, dependency-light (fits the 5-min / 16 GB / CPU box).
- Claude (offline only) — JD→rubric parsing and eval labeling; never called at rank-time, no candidate data sent to any API.
- pytest (36 tests), Streamlit (HF sandbox), python-pptx + matplotlib (this deck).

- GitHub: https://github.com/jacklachan/lighthouse-indiaruns
- Live sandbox (HF Spaces): https://huggingface.co/spaces/jacklachan/lighthouse
- Reproduce: pip install -r requirements.txt → python precompute.py → python rank.py → python validate_submission.py submission.csv
- Demo video: [placeholder — add link before submission]
- AI use: Claude offline for JD parsing + eval labeling + coding assistant; no candidate data hit any API at rank-time (see submission_metadata.yaml).

Github videolete

redrob | H2S HACK2SKILL

INDIA.RUNS

Build what next India runs on

THANK YOU